

UCS503 - Software Engineering

Prudence Money

Spend Prudently

Submitted by:

1024030151 Ayush Jain
1024030153 Kevin Thomas Mathew
1024030162 Tanmay Gupta

Group: 3C16 BE Third Year, CSE

Submitted to:

Ms Paramveer Sidhu

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	5
1.1	Background and Context	5
1.1.1	Problem Statement	5
1.2	Project Objectives	5
2	Methodology and Design	7
2.1	System Architecture	7
2.1.1	High-Level Design	7
2.1.2	Technical Stack	8
2.2	Implementation Details	8
2.2.1	Module 1: Core Functionality	8
2.2.2	Module 2: Secondary Features	9
3	Results and Evaluation	11
3.1	Testing Methodology	11
3.2	Final Validation Results	11
3.3	Analysis	11
4	Conclusion	13
4.1	Summary of Work	13
4.2	Key Findings	13
4.3	Future Work and Recommendations	13
4.4	Final Remarks	14
	Bibliography	15

Introduction

1.1 Background and Context

PrudenceMoney is a smart finance-tracking application designed to simplify expense management across UPI, bank transactions, receipts, bills, and informal payments. It reduces manual effort through OCR, automated recurring payment tracking, and smart expense categorization. The application also provides spending analytics to help users understand their financial habits and manage their expenses effectively.

1.1.1 Problem Statement

- **Manual Effort:** Recording and analyzing expenses manually is time-consuming and often inaccurate.
- **Lack of Accountability:** Users may overlook small, frequent expenses, leading to poor spending habits.
- **Multiple Payment Modes:** Tracking cash, UPI, bank transfers, and billed payments in one place is difficult.
- **Increased Spending:** Convenient online shopping has reduced spending friction, increasing the frequency of daily transactions.

1.2 Project Objectives

1. **Objective 1:** Setting a foundation for the OCR pipeline with an enforced image acquisition process. This ensures that images taken by the user satisfy certain minimum criteria such as illumination, blur absence and sufficient spatial coverage.
2. **Objective 2:** Build a User Dashboard for manual entry of transactions and to view their past history

Methodology and Design

2.1 System Architecture

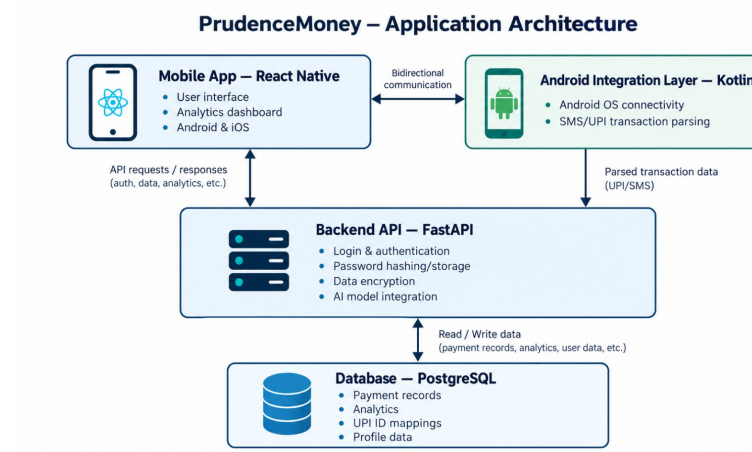


Figure 2.1: Architecture

2.1.1 High-Level Design

- **User Authentication:** Implemented login and sign-up functionality through a temporary Streamlit dashboard, allowing users to access the application and register their accounts.
- **Manual Transaction Entry:** Enabled users to manually record financial transactions, providing an alternative for logging expenses when automated extraction is unavailable.
- **OCR-Based Transaction Recording:** Integrated Optical Character Recognition (OCR) to extract relevant transaction details from uploaded receipts and bills, reducing the need for manual data entry.
- **Transaction Details Display:** Developed a dashboard to display recorded transactions and OCR-extracted information in an organized format, allowing users to review their financial records conveniently.

- **Unified Dashboard:** Combined authentication, transaction recording, and expense viewing into a single interface to demonstrate the application's core functionality and provide a foundation for future frontend development.

2.1.2 Technical Stack

- **Framework/Language:** Python, ReactNative, Kotlin
- **Database:** PostgreSQL
- **Tools:** Ultralytics in-built tools, Custom Python functions
- **Libraries:** OpenCV, PaddleOCR, Ultralytics

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

OCR Foundation : Image Acquisition and Extraction

Goal: Establishing a pre-defined process to ensure that photos taken by the user satisfy a set of certain minimum criteria which ensure a smoother and more accurate extraction process afterwards. Each of these criteria currently focused upon are as follows:

- **Receipt Detection:** Our first priority is to ensure that the current camera view contains a receipt. For this we have used a light weight object detection model, namely *yolo26s*(around 20MB weights), with transfer learning on a dataset [1] specific to receipt detection. Transfer Learning Details are as follows:
 - *Total Images* : 3731
 - *Precision* : 0.994
 - *Recall* : 0.983
 - *mAP50* : 0.992
- **Receipt Coverage:** To ensure that the bill is sufficiently big we set a minimum coverage of **45%** for the bill's *bounding box*, with respect to the image, to ensure that users don't take far fetched images causing the OCR to get a very low resolution input.
Current Coverage Threshold : Minimum 45%

- **Receipt Blur:** This metric measures the existence of motion blur and low resolution text. Even the best OCR model will perform poorly with motion blur. To measure this metric we use variance of the *Laplacian* map of the image.

Initially we curated a custom dataset of 50 text images, with a 50-50 split of blur vs clear. By calculating the laplacian variance of each image and plotting them we got a rough idea of a threshold.

This threshold when implemented seemed to be quite excessive, hence further fine-tuning was done, using a phone's live feed under various blur conditions. This threshold was more appropriate.

Current Blur Threshold : Minimum 400

- **Receipt Illumination:** It is also necessary to ensure that user's doesn't use too dark or bright conditions that may cause undesired artifacts for the OCR model. To measure this, we divide the bill's bounding box into 8 equal sections, the mean intensity of each section (in grayscale) is measured and the mean of those 8 means is our final metric. Please note that the current limits for this metric are **likely** to change in the future.

Current Illumination Threshold : Range 115 - 243

2.2.2 Module 2: Secondary Features

- **Transaction Dashboard:** Developed a temporary dashboard using Streamlit to provide a centralized interface for viewing and managing financial transactions.
- **Transaction History:** Enabled users to view previously recorded transactions along with their relevant details, making it easier to review past expenses.
- **Adding New Transactions:** Provided functionality to add new transactions manually or extract transaction details from uploaded receipts and bills using OCR.
- **Transaction Overview:** Displayed transaction information in an organized format, allowing users to conveniently review their financial records and monitor recorded expenses.

Results and Evaluation

3.1 Testing Methodology

Testing the OCR Pipeline

- **Receipt Detection :** For this we tested on a validation set, part of the large scale receipt detection dataset used for training. The results are displayed later.
- **Integration Testing :** After completing the *Acquisition Process* we integrated PaddleOCR with the extracted bill patch, the generated JSON file is saved to a set directory.
- **Threshold testing :** This is fundamental to our pipeline, hence for testing this we used simple real-world environments under a variety of lighting conditions with commercial bills.
Blur : Tested under White and a combination of White and Yellow lights. Within each lighting category we tested for thresholds at two ranges of spatial coverage namely : Range 1 (45%-55%) and Range 2 (55% +). After testing the blur values in these 4 cases we settled for a final threshold.
Illumination : Used three lighting conditions : Dark, Medium (shadow in background) and Bright (direct light impact). These were based on visual cues. As mentioned prior, limits obtained here are prone to change due to the need for more robustness.
 The **Coverage** and **Blur** thresholds seem to be accurate.
- **Performance Testing :** For OCR, our primary metric of performance is two fold : inference speed and detection accuracy. From the samples we have used to fine tuning thresholds, PaddleOCR's average confidence was always 0.9%+, however, to get a more overall estimate, for the sake of completeness, we wish to test this on a real receipt OCR dataset in the near future.

3.2 Final Validation Results

3.3 Analysis

Analyze your results:

- We have met the objective of setting a foundational **OCR** pipeline

Metric	Result
Training Epochs	8
Training Duration	1.32 hours
Total Validation Images	3731
Precision	0.994
Recall	0.983
mAP@50	0.992
mAP@50-95	0.961
Preprocessing Time	0.2 ms/image
Inference Time	3.7 ms/image
Loss Calculation Time	0.0 ms/image
Postprocessing Time	1.2 ms/image

Table 3.1: Final validation performance of the fine-tuned YOLO model

- A **User Dashboard** for manual entry is set up
- In terms of expectations, we feel the OCR extraction process have better results than expected, this taught us an important lesson about model selection (switch from EasyOCR to PaddleOCR)
- One of our current challenges is simulating the significantly large possibilities of lighting conditions that one can encounter in real life. This explains why some of our thresholds (eg Illumination) are still possibly inaccurate
- Rigorous testing in varying environments, which we intend to do more of, is the main means of tackling the above.

Conclusion

4.1 Summary of Work

- We have set up an enforced image acquisition pipeline for OCR involving receipt detection and condition checking, this is followed by patch extraction using the receipt's detected bounding box which we then pass through our OCR model.
- The model internally performs orientation correction and unwarping prior to OCR, the final extracted text is stored as a JSON file.
- A User Dashboard has been set up for manual user entry in difficult scenarios

4.2 Key Findings

- **Finding 1:** Learned that much more testing is needed to simulate the vast variety of lighting conditions and receipt types
- **Finding 2:** Need to dedicate more time and effort to developing the User Interface, given the learning curve involved
- **Finding 3:** Estimating thresholds for an appropriate OCR setup was much harder than expected and is to be worked upon more

4.3 Future Work and Recommendations

Discuss how others could extend or improve your work:

1. Currently patch extraction is automatic, this will be made user-controlled later on
2. Currently we have only tested for single bill environments, after finding appropriate lighting thresholds, will move on to multi-bill setups
3. The metric for illumination was learned to be ineffective in certain lighting conditions (uneven distribution), hence we consider integrating concepts such as *variance* to this metric,
4. Current prototype does not integrate OCR with user's dashboard in the same manner as intended final product, that will be a top priority

5. As mentioned prior, we need to test our OCR model on large scale data to give a value to its performance, something not done yet due to confidence in its inference capability and focus on other areas.
6. After this we move onto SMS integration for Android Users
7. Finally we will focus on integrating the frontend, backend and OCR into a uniform system with user analytics

4.4 Final Remarks

We learned the need for more emphasis on all round integration in addition to more environmental testing to make our product robust to any user. In addition , the value of initially selecting a correct model has been highlighted.

Keeping the above in mind, we move on to further advancements.

Bibliography

- [1] Xinli Teo. *Receipt Detection Dataset for YOLO*. Version 3. Mendeley Data, 2026 (cit. on p. 8).